

02 — EXPLAIN ANALYZE Deep Dive

"EXPLAIN without ANALYZE is like a weather forecast without looking outside."

Table of Contents

1. [Learning Objectives](#)
 2. [Actual vs Estimated Rows](#)
 3. [Timing Fields](#)
 4. [BUFFERS Option Deep Dive](#)
 5. [ASCII Diagram: Annotated EXPLAIN ANALYZE Output](#)
 6. [Loops and Multiple Executions](#)
 7. [Sort Method Details](#)
 8. [Hash Join Memory Details](#)
 9. [Parallel Query in EXPLAIN ANALYZE](#)
 10. [Progressive EXPLAIN \(pg_query_plan\)](#)
 11. [FORMAT JSON for Tooling](#)
 12. [pg_stat statements Integration](#)
 13. [Common Diagnosis Patterns](#)
 14. [Common Mistakes](#)
 15. [Best Practices](#)
 16. [Interview Questions & Answers](#)
 17. [Exercises with Solutions](#)
 18. [Production Scenarios](#)
 19. [Cross-References](#)
-

Learning Objectives

After completing this section you will be able to:

- Compare actual vs estimated rows to diagnose statistics problems
 - Interpret timing fields to find the most expensive nodes
 - Read buffer statistics to diagnose I/O vs CPU bottlenecks
 - Understand loops and how to interpret cumulative times
 - Use FORMAT JSON with external analysis tools
 - Build a systematic diagnostic workflow for slow queries
-

Actual vs Estimated Rows

The most important diagnostic comparison in EXPLAIN ANALYZE is: **estimated rows (planner) vs actual rows (execution)**

Why Estimates Matter

PostgreSQL's cost-based optimizer uses estimated row counts to choose between plan alternatives. If estimates are wrong, the chosen plan may be suboptimal:

- Underestimate → optimizer thinks fewer rows → prefers Nested Loop (fast for few rows, slow for many)

- Overestimate → optimizer thinks more rows → prefers Hash Join (fast for many rows, unnecessary for few)

Perfect Estimates

```
Seq Scan on orders (cost=0.00..289.00 rows=22312 width=8)
      (actual time=0.034..2.789 rows=22312 loops=1)
           ↑ estimate           ↑ actual
```

Estimate = 22,312, actual = 22,312. Statistics are accurate.

Underestimate (Danger!)

```
Index Scan on orders (cost=0.43..8.45 rows=5 width=128)
      (actual time=0.056..456.789 rows=50000 loops=1)
```

Planner thought 5 rows, got 50,000. It chose an Index Scan expecting to fetch 5 heap pages — but had to fetch 50,000. A Seq Scan would have been faster.

Fix: `ANALYZE orders;` — refresh statistics.

Overestimate (Less dangerous but still wrong)

```
Hash Join (cost=123.45..9876.54 rows=100000 width=24)
      (actual time=2.345..5.678 rows=45 loops=1)
```

Planner built a huge hash table expecting 100,000 matches — wasted memory and time.

Fix: `ANALYZE` both tables; check for data correlation issues.

When ANALYZE Doesn't Help: Extended Statistics

Some estimation problems persist even after ANALYZE because the planner assumes column independence. For correlated columns:

```
-- Example: city and zip_code are highly correlated
-- Planner estimates: P(city='NYC') × P(zip='10001') = 0.01 × 0.001 = 0.000001 (way
too low)
-- Actual: if city='NYC' then zip='10001' is common

-- Solution: create extended statistics
CREATE STATISTICS stats_city_zip ON city, zip_code FROM addresses;
ANALYZE addresses;
-- Now planner estimates the joint distribution correctly
```

Timing Fields

```
Node (cost=...) (actual time=startup_time..total_time rows=N loops=M)
```

- `startup_time` : ms until first row returned (e.g., Sort: must read all input first)

- `total_time` : ms when last row returned from this node
- `loops` : how many times this node was executed

Critical: times are PER LOOP for inner nodes. Total time = time × loops.

```
Hash Join (actual time=5.678..35.123 rows=50123 loops=1)
-> Index Scan on users (actual time=0.056..3.456 rows=7890 loops=1)
-> Hash (actual time=4.567..4.567 rows=22312 loops=1)
    -> Seq Scan on orders (actual time=0.034..2.789 rows=22312 loops=1)
```

Total query time is the top node's `total_time`: 35.123 ms.

But note: the inner side of a nested loop runs once per outer row:

```
Nested Loop (actual time=0.034..9876.543 rows=5000 loops=1)
-> Seq Scan on departments (actual time=0.023..0.056 rows=50 loops=1)
-> Index Scan on employees (actual time=0.089..3.456 rows=100 loops=50)
    ↑ loops=50, runs once per department
    ↑ total cost for Index Scan = 3.456 ms × 50 = 172.8 ms
```

The reported `actual time` for the inner node is per-iteration. Total contribution = `total_time` × `loops` .

BUFFERS Option Deep Dive

`BUFFERS` shows page-level I/O statistics for every node.

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM orders WHERE customer_id = 42;
```

```
Index Scan using idx_orders_cust on orders
(actual time=0.034..1.234 rows=50 loops=1)
Index Cond: (customer_id = 42)
Buffers: shared hit=8 read=3
```

Buffer Types

Type	Description	Performance
shared hit=N	N pages found in shared_buffers (PostgreSQL cache)	Fast (in-memory)
shared read=N	N pages read from OS (disk or OS file cache)	Slow if cold
shared dirtied=N	N pages modified in shared_buffers	Normal for writes
shared written=N	N pages written from shared_buffers to disk	Background write
local hit=N	N temp table pages found in local buffer cache	
local read=N	N temp table pages read	
temp read=N	N temp file pages read (spill to disk!)	Very slow
temp written=N	N temp file pages written (spill to disk!)	Very slow

Diagnosing I/O vs Cache

```
Seq Scan on orders (actual time=0.034..2.789 rows=22312 loops=1)
  Buffers: shared hit=156                                ← all in cache (fast)
```

vs

```
Seq Scan on orders (actual time=5.034..2789.123 rows=22312 loops=1)
  Buffers: shared hit=10 read=146                        ← mostly disk reads (slow)
```

Identifying Disk Spill (work_mem Issue)

```
Hash (actual time=45.678..45.678 rows=500000 loops=1)
  Buckets: 131072 Batches: 8 Memory Usage: 4096kB      ← Batches>1 = disk spill!
  Buffers: shared hit=2345, temp read=1234 written=1234 ← temp files used
  -> Seq Scan on large_table (...)
```

Batches: 8 means the hash table didn't fit in work_mem — it was split into 8 batches and processed 8 times. Increase `work_mem` to eliminate batching:

```
SET work_mem = '256MB'; -- or SET work_mem = '1GB' if needed
```

ASCII Diagram: Annotated EXPLAIN ANALYZE Output

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT p.name, COUNT(o.id) AS order_count, SUM(o.total) AS revenue
FROM products p
LEFT JOIN order_items oi ON p.id = oi.product_id
LEFT JOIN orders o ON oi.order_id = o.id
WHERE o.status = 'completed' OR o.id IS NULL
GROUP BY p.id, p.name
HAVING COUNT(o.id) > 0
ORDER BY revenue DESC
LIMIT 10;
```

```
Limit                                ← [TOP NODE] takes first 10
(cost=12345.67..12345.69 rows=10 width=32)
(actual time=234.567..234.568 rows=10 loops=1)
├──
└── Sort                              ← ORDER BY revenue DESC
    Sort Key: (sum(o.total)) DESC      ← what's being sorted
    Sort Method: top-N heapsort Memory: 28kB ← in-memory (good), top-N efficient
    (actual time=234.565..234.566 rows=10)
    ├──
    └── HashAggregate                  ← GROUP BY + HAVING
        Group Key: p.id, p.name
        Batches: 1 Memory Usage: 892kB  ← fits in work_mem (good)
        Filter: (count(o.id) > 0)        ← HAVING clause applied
```

```

Rows Removed by Filter: 2340      ← 2340 products had 0 orders
(actual time=210.234..232.456 rows=7660)
|
└─ Hash Left Join                  ← JOIN order_items to orders
    Hash Cond: (oi.order_id = o.id)
    (actual time=5.678..189.123 rows=890456 loops=1)
    Buffers: shared hit=1234 read=456
    |
    └─ Hash Left Join              ← JOIN products to order_items
        Hash Cond: (p.id = oi.product_id)
        (actual time=3.456..45.678 rows=890456 loops=1)
        |
        └─ Seq Scan on products    ← read all products
            (rows=10000 → actual=10000) ← estimate matches ✓
            Buffers: shared hit=89
            |
            └─ Hash                ← hash table of order_items
                Buckets: 1048576
                Batches: 1  Memory Usage: 45678kB ← 45 MB! large hash
                → Seq Scan on order_items
                    (rows=900000 → actual=890456) ← good estimate ✓
                    Buffers: shared hit=4567 read=0 ← all cached
            |
            └─ Hash                ← hash table of completed orders
                Buckets: 131072
                Batches: 1  Memory Usage: 8192kB
                → Seq Scan on orders
                    Filter: (status = 'completed')
                    Rows Removed by Filter: 45678 ← many non-completed
orders
                    (rows=234567 → actual=234567) ← but estimate right
                    Buffers: shared hit=2345

Planning Time: 5.678 ms
Execution Time: 234.890 ms

ANNOTATIONS:


---


✓ GOOD: All estimates match actuals → statistics are current
✓ GOOD: No disk spill (Batches: 1 everywhere)
✓ GOOD: Sort uses top-N heapsort (efficient for LIMIT)
⚠ WARNING: 45 MB hash table for order_items – if work_mem < 45 MB, this would spill
⚠ WARNING: 45678 rows removed by Filter on orders – index on orders(status) would help
? OPPORTUNITY: Consider partial index on orders WHERE status='completed'

```

Loops and Multiple Executions

In Nested Loop joins, the inner relation is scanned once per outer row. The reported time for inner nodes is per-iteration.

```

Nested Loop (actual time=0.056..567.234 rows=150 loops=1)
-> Index Scan on customers (actual time=0.034..0.045 rows=3 loops=1)
    Index Cond: (type = 'premium')
-> Index Scan on orders (actual time=0.089..3.456 rows=50 loops=3)
    Index Cond: (customer_id = customers.id)
    Buffers: shared hit=15 read=0

```

Inner node `orders` ran 3 times (loops=3, once per customer row):

- Per-iteration time: 3.456 ms
- Total time: 3.456 ms × 3 = 10.368 ms
- Per-iteration buffers: shared hit=15
- Total buffers: 15 × 3 = 45 pages

When analyzing performance, multiply inner node times by their loop count.

Sort Method Details

```

Sort (actual time=456.789..512.345 rows=100000 loops=1)
  Sort Key: created_at DESC
  Sort Method: ???

```

Sort Method	Description	Memory	Performance
quicksort	In-memory quicksort	work_mem	Fast
top-N heapsort	Only keeps top N (for LIMIT)	Small	Fast
external sort	Disk-based (2-pass)	Exceeds work_mem	Slow
external merge	Disk-based multi-pass	Much exceeds work_mem	Very slow

Fix for disk sort:

```

-- Session-level (safe)
SET work_mem = '256MB';

-- Or: restructure to use an index that provides pre-sorted order
CREATE INDEX ON orders(tenant_id, created_at DESC);
-- Query: WHERE tenant_id = ? ORDER BY created_at DESC LIMIT 10
-- Now uses Index Scan in order → no Sort node at all!

```

Hash Join Memory Details

```

Hash (actual time=12.345..12.345 rows=50000 loops=1)
  Buckets: 65536 Batches: 1 Memory Usage: 2453kB
-> Seq Scan on customers (rows=50000)

```

Field	Meaning
-------	---------

Buckets: 65536	Number of hash buckets allocated
Batches: 1	All data fits in work_mem — no spill
Batches: N>1	Data spilled to disk in N batches — increase work_mem
Memory Usage: NkB	Peak memory used for this hash table

Required work_mem: $\text{rows} \times \text{width} \times 1.2 / 1024$ KB approximately. For 50,000 rows $\times$ 40 bytes wide $\times$ 1.2 overhead = ~2.4 MB.

Parallel Query in EXPLAIN ANALYZE

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT COUNT(*) FROM large_table WHERE condition = true;
```

```
Finalize Aggregate (actual time=234.567..234.568 rows=1 loops=1)
-> Gather (actual time=234.456..234.512 rows=3 loops=1)
    Workers Planned: 2
    Workers Launched: 2
    -> Partial Aggregate (actual time=232.345..232.346 rows=1 loops=3)
        -> Parallel Seq Scan on large_table
            (actual time=0.023..189.234 rows=3333333 loops=3)
            Workers: 2 (but leader worker also scans)
            Buffers: shared hit=12345 read=4567
```

- Workers Planned: 2 — planner wanted 2 workers
- Workers Launched: 2 — 2 were actually started
- loops=3 — executed 3 times (2 workers + 1 leader)
- rows=3333333 loops=3 — each worker processed ~3.3M rows (10M total)

FORMAT JSON for Tooling

For complex queries, use JSON format with online tools:

```
EXPLAIN (ANALYZE, BUFFERS, FORMAT JSON) your_query;
```

Useful tools:

- explain.depesz.com — paste JSON/text, get visual breakdown
- explain.tensor.ru — alternative with node timing visualization
- [pev2](#) — modern visual explain viewer
- auto_explain extension — automatically logs slow query plans to PostgreSQL log

auto_explain Setup

```
-- Enable auto_explain (logs plans for slow queries)
LOAD 'auto_explain';
SET auto_explain.log_min_duration = 1000; -- log queries > 1 second
```

```
SET auto_explain.log_analyze = on;
SET auto_explain.log_buffers = on;
SET auto_explain.log_format = 'json';
-- Plans appear in PostgreSQL log file
```

pg_stat_statements Integration

`pg_stat_statements` tracks query execution statistics across all sessions.

```
-- Enable pg_stat_statements
CREATE EXTENSION IF NOT EXISTS pg_stat_statements;

-- Find the slowest queries
SELECT
    LEFT(query, 100) AS query_snippet,
    calls,
    ROUND(mean_exec_time::numeric, 2) AS mean_ms,
    ROUND(total_exec_time::numeric / 1000, 2) AS total_sec,
    ROUND(stddev_exec_time::numeric, 2) AS stddev_ms,
    rows,
    ROUND(100.0 * blk_read_time / NULLIF(total_exec_time, 0), 2) AS pct_io_time
FROM pg_stat_statements
WHERE calls > 10
ORDER BY total_exec_time DESC
LIMIT 20;

-- Find queries with high variance (inconsistent, sometimes slow)
SELECT
    LEFT(query, 100) AS query,
    calls,
    mean_exec_time,
    stddev_exec_time,
    ROUND(stddev_exec_time / NULLIF(mean_exec_time, 0), 2) AS cv -- coefficient of
variation
FROM pg_stat_statements
WHERE calls > 100
ORDER BY cv DESC
LIMIT 10;
```

After finding a slow query, run EXPLAIN ANALYZE on it directly.

Common Diagnosis Patterns

Pattern 1: Stale Statistics (estimate vs actual mismatch)

```
Seq Scan  (rows=5 width=128)
         (actual rows=50000 loops=1)
```

Action: `ANALYZE table_name;` **If still wrong:** `ALTER TABLE t ALTER COLUMN c SET STATISTICS 500;`
`ANALYZE t;` **If still wrong** (correlated columns): `CREATE STATISTICS;` `ANALYZE;`

Pattern 2: Missing Index

```
Seq Scan on orders (rows=1000000)
  Filter: (customer_id = 42)
  Rows Removed by Filter: 999995
```

Action: `CREATE INDEX CONCURRENTLY ON orders(customer_id);`

Pattern 3: work_mem Insufficient

```
Sort Method: external merge Disk: 102400kB
or
Hash Batches: 16 Memory Usage: 4096kB
or
Buffers: temp read=1234 written=1234
```

Action: `SET work_mem = '256MB';` (session-level) Or: restructure query to avoid sort/hash, use indexes

Pattern 4: Wrong Join Type (Nested Loop on large tables)

```
Nested Loop (rows=1000000 loops=1)
  Seq Scan on orders (rows=1000000)
  Index Scan on customers (loops=1000000)
```

Action: This suggests wrong join order. Check join conditions. Force plan change: `SET enable_nestloop = off;`

Pattern 5: Too Many Heap Fetches in Index-Only Scan

```
Index Only Scan (rows=50000)
  Heap Fetches: 49987
```

Action: `VACUUM table_name;` — sets all-visible bits in visibility map.

Common Mistakes

1. Forgetting DML Runs with EXPLAIN ANALYZE

```
-- This will actually run the DELETE and may commit without BEGIN:
EXPLAIN ANALYZE DELETE FROM orders WHERE status = 'old';
-- ALWAYS wrap DML in BEGIN/ROLLBACK
```

2. Comparing plans from different time periods

A plan that runs 100ms at 2am may run 5 seconds at 9am due to lock contention, cache pressure, or different data. Always compare apples-to-apples.

3. Ignoring inner node times for Nested Loops

The inner node's time must be multiplied by loops count.

4. Not enabling BUFFERS

Without BUFFERS, you can't determine if slowness is I/O or CPU.

Best Practices

1. Use `(ANALYZE, BUFFERS)` **always** for slow query diagnosis.
 2. Check all nodes, not just the top — slowness is usually at leaf nodes.
 3. Calculate total inner-node time by multiplying per-iteration time × loops.
 4. Check `temp read/written` — any non-zero value is a performance problem.
 5. Compare plan to expected plan — know what you expected before seeing what you got.
 6. Run **EXPLAIN ANALYZE multiple times** — first run may show cold-cache behavior; subsequent runs show warm-cache behavior (both are valid scenarios to test).
 7. Use **JSON format + tools** for complex 10+ node plans.
-

Interview Questions & Answers

Q1. What is the difference between `rows` and `actual rows` in EXPLAIN ANALYZE?

A: `rows` is the planner's statistical estimate of how many rows this node will return, computed before execution using statistics from `pg_statistic`. `actual rows` is the true count measured during execution. Large discrepancies (more than 10×) indicate stale statistics (fix: ANALYZE) or data distribution that the planner's statistical model doesn't capture well (fix: extended statistics, increase statistics target). When estimates are wrong, the planner may choose a suboptimal plan — this is the primary cause of unexplained slow query regressions after data changes.

Q2. What does `Batches: 8` mean in a Hash node and how do you fix it?

A: `Batches: 8` means the hash table didn't fit in `work_mem` and was divided into 8 batches. PostgreSQL builds the hash table for the first batch, probes it with the build side, then repeats 7 more times for the remaining batches. This 8× additional processing dramatically slows the hash join. To fix: increase `work_mem` (`SET work_mem = '512MB'`) until Batches becomes 1. Alternatively, restructure the query to avoid the large hash join (add more selective filters earlier, use materialized pre-aggregation).

Q3. How do you determine which node in EXPLAIN ANALYZE is taking the most time?

A: Start from the top node's total actual time and subtract child node times. The difference is the time this specific node's processing took (vs its children's contribution). For nested loops, multiply inner node time by loops count. Look for: (1) nodes with large `actual time` differences between startup and total, (2) Sort nodes showing disk methods, (3) Hash nodes with Batches > 1, (4) any node with `temp read/written` in buffers.

Exercises with Solutions

Exercise 1

Analyze this EXPLAIN ANALYZE output and identify the problem:

```
Nested Loop  (cost=0.43..987654.32 rows=500000 width=64)
    (actual time=0.056..89234.567 rows=450000 loops=1)
  -> Seq Scan on products  (rows=5000 actual rows=5000 loops=1)
      Buffers: shared hit=45
  -> Index Scan on order_items  (rows=100 actual rows=90 loops=5000)
      Index Cond: (product_id = products.id)
      Buffers: shared hit=3 read=1 (per iteration)
```

Solution:

- **Problem:** Nested Loop with `loops=5000` on the inner Index Scan
- **Inner node total time:** ~(estimated from total - outer) = very high
- **Total buffer reads:** 1 page × 5000 loops = 5000 random disk reads
- **Better plan:** Hash Join or Merge Join (process all order_items once)
- **Fix:** `SET enable_nestloop = off;` to test, then investigate why planner chose NL
 - Possible cause: planner estimated inner rows=100 (low → NL preferred), actual=90 (accurate but timing is still wrong because 5000 loops of random I/O is slow)
 - Solution: Increase `work_mem` for Hash Join, or check `effective_cache_size` setting

Production Scenarios

Scenario 1: Nightly Report Taking 45 Minutes

A nightly report query was taking 45 minutes. EXPLAIN ANALYZE (run in a dev environment with a data copy) showed:

```
Sort  (Sort Method: external merge  Disk: 2048000kB)  ← 2 GB disk sort!
HashAggregate  (Batches: 64  Disk Usage: 4096000kB)  ← 4 GB hash disk spill!
Hash Join  (Batches: 32)
```

Root cause: default `work_mem` = 4MB was completely inadequate for the 10M+ row aggregation.

Fix:

```
-- For this specific nightly report session:
SET work_mem = '2GB';
SET max_parallel_workers_per_gather = 4;
-- Re-run query: 45 minutes → 3 minutes
```

Cross-References

- [01 explain basics.md](#) — EXPLAIN basics and node types
- [03 scan types.md](#) — Scan types in detail
- [04 join algorithms.md](#) — Join algorithms and memory usage

- [05_query_planner.md](#) — How estimates are computed
- [06_statistics_and_estimates.md](#) — pg_statistic and ANALYZE
- [09_slow_query_analysis.md](#) — pg_stat_statements workflow